ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

COLLEGE OF ELECTRICAL ENGINEERING AND COMPUTING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Programming Languages (CSEg4306)

አማርኛ ፕሮግራሚንግ ቋንቋ (AML)

"Local Language Programming for Beginners"

Educational Compiler Project — Source-to-Source Translation

No.	Name	ID	Section
1	Natnael Yilma	UGR/31044/15	1
2	Temesegen Mekonnen	UGR/31277/15	1
3	Abdulhalim Aliye	UGR/30032/15	1
4	Abdulwedud Yassin	UGR/30038/15	1
5	Kenenisa Beyan	UGR/30772/15	2

Executive Summary

This report presents AML (አማርኛ ፕሮግራሚንግ ቋንቋ), a beginner-oriented programming language whose keywords and control structures are expressed in Amharic, together with a Python-based source-to-source translator, validated test programs, and a concise analysis of design decisions and challenges inherent in local-language compiler construction.

AML targets Ethiopian learners who are more comfortable reading Amharic than English when learning foundational concepts such as variables, conditionals, loops, and functions. Programs are written in UTF-8 source files, processed through a classical lexer–parser–code generator pipeline, and emitted as executable Python. A PyQt6 desktop editor (amharic_editor/) provides an optional integrated environment with syntax highlighting, translation, and run support.

Three representative programs—variables and arithmetic, conditional control flow, and a feature demonstration combining while loops and functions—were translated and executed successfully. Verified stdout matches expected educational outcomes (e.g., sum 30, Amharic branch message, loop 0–4, function result 8).

1. Introduction

Programming education worldwide often assumes English literacy. For many Ethiopian students, English adds cognitive load on top of already unfamiliar abstractions (variables, memory, control flow). AML reduces that barrier by using Amharic keywords that read like natural instructions for example, ከሆነ (“if it is so”) and መጨረሻ (“end”) while preserving familiar mathematical operators (+, ==) and translating to widely available Python runtimes.

1.1 Objectives

- Define a small, teachable language with documented keywords, syntax, and examples.
- Implement a command-line translator: python translator.py program.aml → program.py.
- Demonstrate correctness through at least three test programs with source, generated code, and execution output.
- Document design rationale, beginner benefits, and Unicode/localization challenges.

1.2 Scope

Version 1.0 includes numeric and text declarations, print and input, if/else, while loops, functions with return, boolean literals, and logical operators. Features such as for-loops (ለ), lists (ዝርዝር), and modules (አስገባ) are reserved for future work as documented in FUTURE_IMPROVEMENTS.md.

2. Background and Context

Local-language programming initiatives aim to align syntax with the learner's everyday language without sacrificing rigor. AML follows the educational transpiler pattern: rather than building a full virtual machine, the project compiles AML to Python so students can run programs immediately and later inspect readable target code.

The project repository organizes the compiler under src/ (lexer, parser, ast_nodes, codegen, errors), exposes translator.py as the CLI entry point, ships example programs under መማሪያ፡ኢት/ (entry examples), and provides extended documentation in docs/ plus an optional amharic_editor/ IDE for classroom use.

2.1 Assumptions

- Source encoding is UTF-8 end-to-end; terminals and editors must support Ethiopic script display.
- Target language is Python 3.8+; generated files use the .py extension adjacent to the source name.
- Assignment requirements reference .yl as a generic local extension; this implementation uses .aml or .፡ኢት per project convention.
- No separate runtime VM is required; Python executes the generated output.

3. Methodology and Implementation Approach

Development followed a specification-first methodology: keywords and grammar were written in LANGUAGE_SPECIFICATION.md, then implemented as a multi-phase compiler. Each phase

was validated incrementally using the --tokens debug flag and example programs before integration into the desktop editor.

Table 1: Translator module responsibilities

Module	Responsibility
src/lexer.py	Unicode-aware tokenization; Amharic keyword table
src/parser.py	Recursive-descent parsing; block boundaries via stop tokens
src/ast_nodes.py	Dataclass AST representation
src/codegen.py	AST → formatted Python
src/errors.py	Amharic syntax error messages with line/column
translator.py	CLI: read source, translate, write .py

3.1 Command-Line Translator

The required command-line interface is implemented in translator.py. The user invokes translation with a single source path; by default the output file shares the input basename with a .py suffix.

Listing 1: Translator usage (from translator.py)

```
python translator.py  program.aml
python translator.py  program.aml -o output.py
python translator.py  program.aml --tokens
```

On success the tool prints a confirmation line (ተተርጉሟል: … → …). Syntax errors raise AmharicSyntaxError with messages such as ስህተት: መጨረሻ ተጠበቀ when a block is not closed with መጨረሻ.

4. Language Specification

The following subsections satisfy the two-to-three-page language specification requirement: keyword semantics, syntax rules, example programs, and beginner-friendly design principles. Full EBNF-style grammar appears in docs/LANGUAGE_SPECIFICATION.md.

4.1 Keywords and Meanings

Table 2: Core AML keywords (v1.0)

English	Amharic	Meaning / usage
print	አትም	Output an expression to the

		console
input	ግቤት	Read user input (mapped to Python input/prompt patterns)
if	ከሆነ	Start conditional block
else	ካልሆነ	Optional else branch
while	እስከ	Loop while condition is true
function	ተግባር	Define a function
return	መልስ	Return a value from a function
end	መጨረሻ	Close block (if, while, function)
true / false	እውነት / ሐሰት	Boolean literals
and / or / not	እና / ወይም / አይደለም	Logical operators
number	ቁጥር	Declare numeric variable
text	ጽሑፍ	Declare string variable

4.2 Syntax Rules

- One statement per line; blocks end with መጨረሻ, not braces.
- Comments begin with # and run to end of line.
- Variable declaration: ቁጥር name = expr or ጽሑፍ name = expr; reassignment uses name = expr.
- Output: አትም expression.
- Conditional: ከሆነ expr … optional ካልሆነ … መጨረሻ.
- Loop: እስከ expr … መጨረሻ.
- Function: ተግባር name(params) … optional መልስ expr … መጨረሻ.
- Operators: arithmetic + - * /; comparison == < >; logical እና ወይም አይደለም; grouping ( ).
- Identifiers may use Amharic, English, or mixed Unicode letters (e.g., ድምር, x).

Listing 2: Conditional syntax template

```
ከሆነ <expression>
   <statements>
ካልሆነ
   <statements>
መጨረሻ
```

4.3 Example Programs (Specification Samples)

Listing 3: Minimal variables and arithmetic (01_variables)

```
ቁጥር x = 10
ቁጥር y = 20
አትም x + y
```

4.4 Beginner-Friendly Design

1. Natural keywords — ከሆነ reads like Amharic prose rather than opaque English tokens.
2. Explicit block endings — መጨረሻ makes scope visible; learners are not required to match invisible braces.
3. Familiar operators — + and == connect to school mathematics and later English-based languages.
4. Simple type hints — ቁጥር and ጽሑፍ clarify intent without a heavy static type system.
5. Amharic error messages — e.g., ስህተት: መጨረሻ ተጠበቀ localizes failure explanations.
6. Real parsing — no fragile text replacement; nested blocks and strings are handled correctly.

5. Test Programs and Results

Three required program categories are demonstrated below with AML source, generated Python (produced by translator.py), and verified execution output. A fourth program (functions) extends the feature demonstration.

5.1 Basic Program — Variables and Mathematics

Listing 4: AML source (01_variables.፡ኢት)

```
# ቀላል ቁጥሮች እና አትም
ቁጥር x = 10
ቁጥር y = 20

አትም x + y
```

Listing 5: Generated Python

```
x = 10
y = 20
print((x + y))
```

Table 3: Execution result — basic program

Step	Command / result
Translate	python translator.py መግቢያ፡ኢት/01_variables.፡ኢት
Execute	python 01_variables.py
Stdout	30

5.2 Control Flow Program — Conditionals

Listing 6: AML source (02_conditionals.፡ኢት)

```
ቁጥር x = 7

ከሆነ x > 5
   አትም "x ትልቅ ነው"
ካልሆነ
   አትም "x ትንሽ ነው"
መጨረሻ
```

Listing 7: Generated Python

```
x = 7
if x > 5:
   print("x ትልቅ ነው")
else:
   print("x ትንሽ ነው")
```

Table 4: Execution result — conditionals (x = 7)

Step	Command / result
Translate	python translator.py መግቢያ፡ኢት/02_conditionals.፡ኢት

Execute	python 02_conditionals.py
Stdout	x ትልቅ ነው

5.3 Feature Demonstration — While Loop and Functions

While loop:

Listing 8: AML source (03_while_loop.ኤት)

```
ቁጥር i = 0

እስክ i < 5
   አትም i
   i = i + 1
መጨረሻ
```

Listing 9: Generated Python (while)

```
i = 0
while i < 5:
   print(i)
   i = i + 1
```

Table 5: While loop execution output

Execute stdout (03_while_loop)
0 1 2 3 4

Functions (additional feature demo):

Listing 10: AML source (04_functions.ኤት)

```
ተግባር ድምር(a, b)
   መልስ a + b
መጨረሻ

ቁጥር ውጤት = ድምር(5, 3)

አትም ውጤት
```

Listing 11: Generated Python (functions)

```
def ድምር(a, b):
   return a + b
ውጤት = ድምር(5, 3)
print(ውጤት)
```

Table 6: Feature demonstration results

Program	Verified stdout

03_while_loop	0, 1, 2, 3, 4 (each on its own line)
04_functions	8

6. Design Decisions and Discussion

6.1 Key Design Decisions

- Recursive-descent parser — maps directly to grammar rules; easy to teach alongside the language spec.
- Stop-token blocks (ካልሆነ, መጨረሻ) — parser uses block_until rather than indentation enforcement.
- Translation to Python — zero custom VM; students use a mainstream runtime available in schools.
- Reject text replacement — only structured lex/parse/codegen prevents false keyword matches in strings.
- Unicode-first — keywords, identifiers, and errors are stored and emitted as UTF-8 text.

6.2 Why AML Helps Beginners

Beginners benefit when reading code feels like reading instructions in their own language. AML lowers the affective filter: students focus on logic (conditions, loops, functions) before memorizing English tokens. Because output is readable Python, instructors can bridge gradually to industry languages. The optional PyQt6 editor adds Amharic UI labels, syntax highlighting, and one-click translate/run—supporting classrooms with limited CLI comfort.

6.3 Challenges in Local-Language Design

Table 7: Local-language and Unicode challenges

Challenge	**Impact**	**Mitigation in AML**
Unicode in tools	Terminals/editors may garble Ethiopic	UTF-8 everywhere; PYTHONIOENCODING documented; editor fonts guide
Keyword naturalness	Awkward phrases confuse more than help	Community review planned; user guide in Amharic
Identifier scripts	Mixed Amharic/ASCII names	Python 3 Unicode identifier rules in lexer
Error localization	Translated messages must stay precise	Dedicated errors.py with line/column
Spec vs. runtime	Mapping ግቤት to input varies by environment	Documented; editor uses python_runner integration

7. Findings and Results

All mandatory test categories passed: arithmetic output (30), correct conditional branch in Amharic, while loop enumeration 0–4, and function call returning 8. The translator completed without errors on representative sources in መሞቢያ፡ኢት/. Token debug mode (--tokens) confirms keywords are classified before parsing, and generated Python preserves Unicode identifiers such as ድምር and ውጤት.

- Specification completeness: keywords, syntax, examples, and pedagogy are documented in docs/LANGUAGE_SPECIFICATION.md.
- Tooling completeness: CLI translator meets python translator.py <file> deliverable.
- Educational tooling: amharic_editor/ extends reach beyond the command line.

8. Limitations and Recommendations

8.1 Limitations (v1.0)

- No for-loop (ለ), lists (ዝርዝር), or import/module system yet.
- No static type checking beyond declaration keywords.
- No source maps from Python tracebacks back to AML line numbers.
- Desktop editor requires separate PyQt6 dependency installation.

8.2 Recommendations

7. Add pytest golden tests for lexer, parser, and codegen on every example file.
8. Implement for-loops and a minimal standard library with Amharic wrapper names.
9. Publish a web playground (edit → translate → run) for schools without local installs.
10. Conduct native-speaker review of keywords and error strings.
11. Provide optional bilingual aliases (English keywords) for transition to Python/JavaScript courses.

9. Conclusion

AML demonstrates that a small, rigorous, Amharic-surfaced language can be implemented as a maintainable transpiler with clear educational benefits. The project delivers the required specification content, working translator.py CLI, three or more validated test programs with generated Python and execution traces, and a concise design analysis of beginner affordances and localization challenges. Future work should expand control structures and tooling while preserving Unicode-correct, parser-based translation.

References

12. [1] Project documentation: docs/LANGUAGE_SPECIFICATION.md, docs/DESIGN_REPORT.md, docs/FUTURE_IMPROVEMENTS.md.
13. [2] Implementation: translator.py; src/lexer.py, src/parser.py, src/codegen.py, src/errors.py.
14. [3] Examples and expected output: መማቢያ፡ኢት/, መማቢያ፡ኢት/expected/README.md.
15. [4] Python Software Foundation. Python 3 documentation — Unicode HOWTO.
16. [5] Aho, A. V., Lam, M., Sethi, R., & Ullman, J. D. Compilers: Principles, Techniques, and Tools (classic lexer/parser reference).

Appendix A — AML to Python Translation Summary

Table A.1: Construct mapping

AML construct	Generated Python
ቁጥር x = 10	x = 10
ጽሑፍ s = "ሰላም"	s = "ሰላም"
አትም expr	print(expr)
ከሆነ … መጨረሻ	if …: (indented block)
ካልሆነ	else:
እስከ … መጨረሻ	while …:
ተግባር f(a)	def f(a):
መልስ expr	return expr

Appendix B — Sample Error Messages

Table A.2: Amharic syntax errors

Situation	Message
Missing መጨረሻ	ስህተት: መጨረሻ ተጠበቀ
Bad token	ስህተት: ያልተጠበቀ ምልክት '…'
Expected expression	ስህተት: ገላጭ ተጠበቀ
Unclosed string	ስህተት: ያልተጠናቀቀ ጽሑፍ

Appendix C — Regenerating This Report

This document is produced programmatically to ensure consistent formatting and reproducible submissions. Install python-docx if needed, then run:

```
pip install python-docx
python report.py
```